Autonomous Driving ROS System
Technical Documentation

Haitao Yu, Haoran Wang, Yonglin Wang, Haodong Zhu, Hao Ji

1 August 2026

Contents

1 System Architecture

1.1 Overall Structure

This project implements a ROS 2 Jazzy autonomous driving system connected to a Unity vehicle simulator. The main processing chain is

Unity Simulator → Perception → Planning → Decision Making → Control → Unity Simulator.

The system follows a prerecorded route containing 381 waypoints instead of recomputing a global route online. Sensor data are received from Unity through TCP, while vehicle control commands are returned to Unity through UDP.

Package	Main Responsibility
`simulation`	Receives vehicle pose, velocity, RGB images, depth images, and IMU data from Unity; performs coordinate conversion; publishes ROS topics; and sends vehicle control commands.
`perception`	Publishes sensor transforms, converts depth images into point clouds, integrates OctoMap, and detects traffic-light states.
`planning`	Records and loads waypoints, selects the next local goal, and generates a smooth trajectory with a velocity profile.
`decision_making`	Detects hazardous situations, evaluates traffic lights and obstacles, and modifies the planned trajectory by slowing down, shifting laterally, or stopping.
`control`	Tracks the selected trajectory using Pure Pursuit and a PI speed controller, and publishes throttle, brake, and steering commands.
`autonomous_driving_bringup`	Starts and connects all packages together with the Unity simulator.

1.2 Main Data Flow

Unity publishes the vehicle pose, velocity, RGB image, depth image, and IMU data. Pose and velocity are used by the planning and control modules. The traffic-light detector uses the RGB camera image and identifies candidate red, yellow, and green regions through color-based image processing and geometric filtering. The depth image and camera calibration parameters are converted into a three-dimensional point cloud, which is used for both map construction and immediate obstacle detection.

The planning module loads the prerecorded route, selects forward waypoints, and generates a local trajectory. The decision-making module combines this trajectory with traffic-light and obstacle information. It may keep the original trajectory, reduce its target velocity, apply a lateral shift, or replace it with a stopping trajectory. The control module tracks the final trajectory and publishes `/car_command`, which is then transmitted to Unity.

1.3 Main Nodes and Interfaces

Node	Function
`/Unity_ROS_message_Rx`	Receives simulator packets and publishes pose, velocity, image, camera information, and IMU data.
`/ROS_Unity_command_Tx`	Sends the latest throttle, steering, and brake commands to Unity at 100 Hz.
`/depth_to_pointcloud`	Back-projects valid depth pixels into a `PointCloud2` point cloud.
`/traffic_light_detector`	Detects red, yellow, and green traffic lights from RGB images using color segmentation, contour filtering, and multi-frame debouncing.

Node	Function
`/goal_selector`	Estimates the current route position and publishes a short local path segment.
`/trajectory_planner`	Generates a Catmull–Rom trajectory and a curvature- and acceleration-constrained velocity profile.
`/forward_obstacle_monitor`	Projects point-cloud obstacles onto the planned path and estimates obstacle distance and relative closing speed.
`/decision_state_machine`	Selects among DRIVE, CAUTION, AVOIDING, TRAFFIC_STOP, EMERGENCY_STOP, and SENSOR_FAULT states.
`/trajectory_follower`	Uses adaptive Pure Pursuit for steering and a PI controller for longitudinal speed control.

1.4 Runtime and Integration Procedure

At runtime, the bringup package starts the communication bridge, perception, planning, decision-making, and control nodes. After the Unity sensor bridge establishes a TCP connection, it continuously receives simulator packets. The control transmission node returns the latest vehicle command to Unity through UDP at a fixed rate. To reduce startup failures caused by process ordering, the unified launch file starts the ROS communication bridge before the Unity process.

The recommended procedure is as follows:

1. Build the workspace using `colcon build` and source the generated setup file.
2. Check that the IP addresses and ports in Unity match the ROS-side configuration.
3. Start the complete system using `ros2 launch autonomous_driving_bringup autonomous_driving.launch.py`.
4. In RViz2, verify the vehicle frame, camera frame, point cloud, OctoMap, local path, and final trajectory.
5. Observe the traffic-light state, obstacle state, decision state, and vehicle commands to confirm that data propagate continuously through the complete closed loop.

Integration should be debugged layer by layer. First, verify that Unity publishes pose and sensor data. Next, inspect the point cloud and traffic-light result. Then confirm that the planning module advances continuously along the prerecorded route. Finally, verify that the final trajectory produced by the decision module is tracked stably by the controller. This layered approach prevents multiple modules from being modified simultaneously when the vehicle behavior is abnormal.

1.5 Core Topics and Fail-Safe Mechanisms

The main inputs and outputs can be grouped into three categories. The first category contains vehicle-state information, including pose, velocity, and IMU data. The second category contains environmental information, including RGB images, depth point clouds, traffic-light states, and obstacle states. The third category contains motion commands, including the local trajectory, final trajectory, and `/car_command`.

The planning module depends only on the vehicle state and the prerecorded route. The decision-making module adds environmental constraints to the planned trajectory. The control module tracks only the final trajectory provided by the decision-making module.

Several fail-safe rules are implemented to prevent a single node failure from directly generating unsafe commands:

- The controller applies full braking when the trajectory is empty, the input has timed out, or the target velocity is approximately zero.
- The decision state machine assigns the highest safety levels to emergency stop and sensor fault, and applies a minimum holding time before leaving these states.

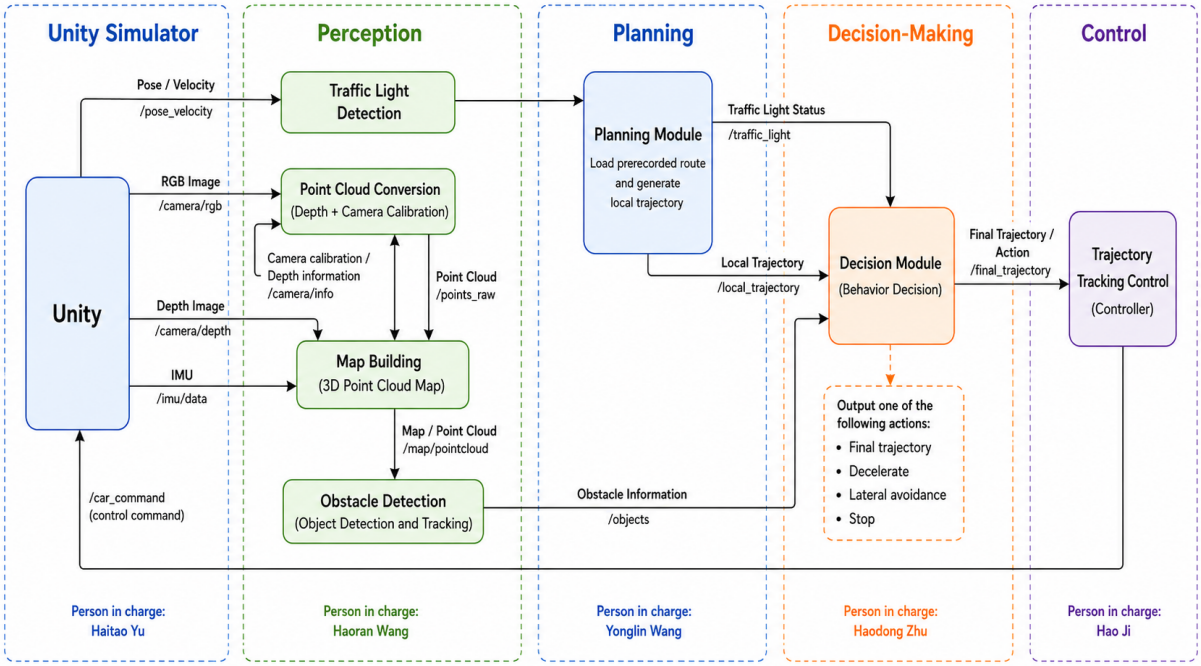

Figure 1: Placeholder for the ROS communication

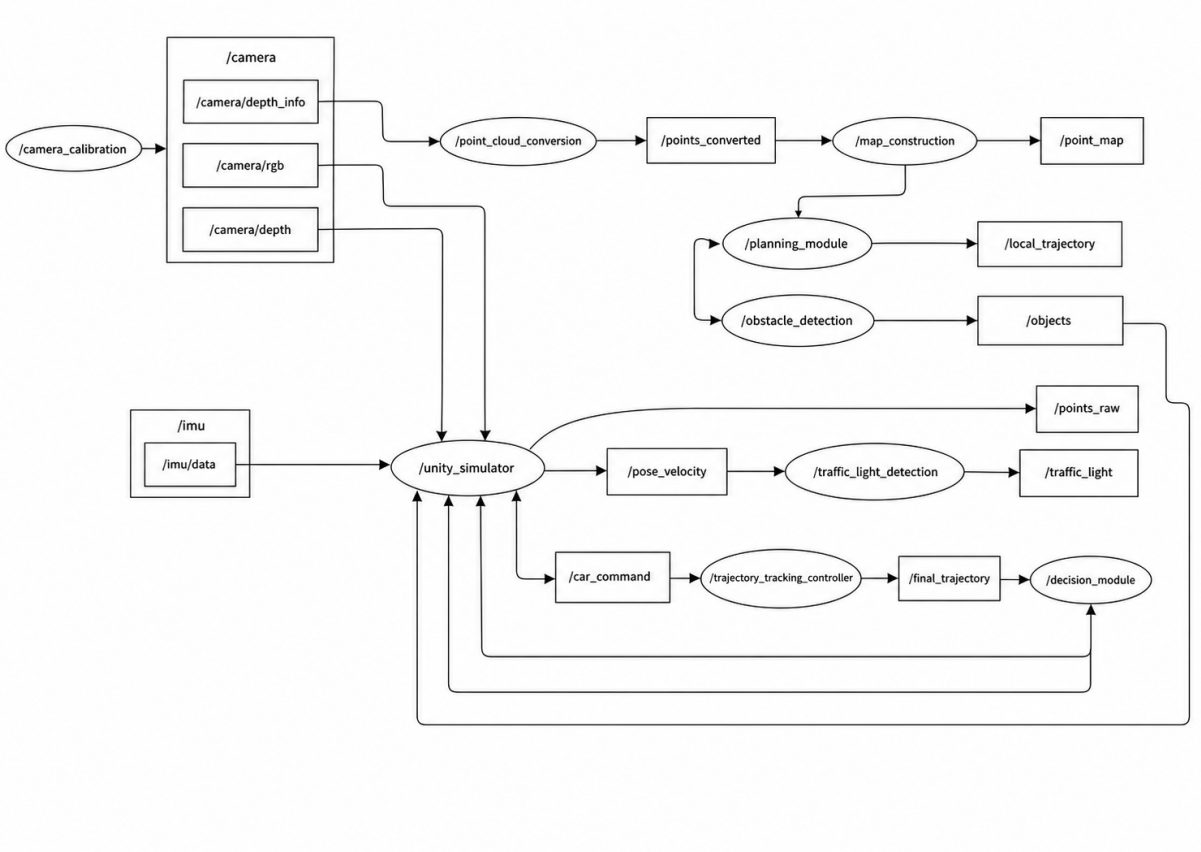

Figure 2: Placeholder for the ROS communication

- The goal selector uses a monotonically increasing route index to prevent the vehicle from jumping back to previously passed waypoints after a local offset.
- Traffic-light detection requires confirmation over several consecutive frames to reduce repeated stop-and-go behavior caused by single-frame noise.

These mechanisms improve basic robustness but do not replace a complete functional-safety concept. The current implementation does not yet include sensor redundancy, unified node-health monitoring, centralized communication-timeout diagnostics, or an independent supervisor for detecting invalid actuator commands.

2 Custom ROS Message and Service Interfaces

The workspace contains five custom `.msg` files distributed across four ROS 2 packages, together with the custom service interface `SetAutonomyEnabled.srv`. The message interfaces exchange project-specific data between simulation, perception, planning, decision making, and control, while the service interface provides a standardized request–response mechanism for changing the autonomous-driving mode [1].

Message	Package	Main Purpose
`VehicleControl`	`simulation`	Contains normalized throttle, steering, and brake commands that are published on `/car_command` and transmitted to the Unity vehicle through UDP. The `reserved` field is currently unused.
`TrafficLightState`	`perception`	Stores the complete RGB-based traffic-light-detection result, including the UNKNOWN/RED/YELLOW/GREEN state, detection flag, confidence value, and candidate pixel area. It is published on `/traffic_light/state`; the decision node currently uses the simplified `/traffic_light/must_stop` topic.
`TrajectoryPoint`	`planning`	Represents one sampled trajectory point with pose, target velocity, curvature, and `time_from_start`. It is used as an element of `Trajectory`.
`Trajectory`	`planning`	Transfers a complete ordered local trajectory from the trajectory planner to the decision state machine and then to the trajectory follower. The decision node may reduce velocities, stop the vehicle, or laterally shift points.
`HazardStatus`	`decision_making`	Describes the nearest relevant obstacle, including distance, closing speed, lateral offset, detection status, and whether a neighboring avoidance corridor is clear.

2.1 Autonomy-Enable Service

The `SetAutonomyEnabled.srv` interface provides a standardized way for an external client, such as a driver interface, monitoring tool, or debugging node, to request that autonomous driving be enabled or disabled. It acts as a safety boundary between an external command and the internal driving system. A request expresses the desired mode, but the system should change its actual state only after checking the relevant safety and system-health conditions.

The service response reports the resulting autonomy state rather than merely repeating the requested value. It can also return a human-readable message explaining whether the change was accepted or why it was rejected. This makes the interface useful for safe mode switching, operator feedback, debugging, and future extensions such as additional sensor-health, fault, or operating-condition checks.

The main custom-message data flow is

$$\begin{aligned} \text{RGB image} &\rightarrow \texttt{TrafficLightState} \rightarrow \text{Decision}, \\ \texttt{PointCloud2} &\rightarrow \texttt{HazardStatus} \rightarrow \text{Decision}, \\ \text{Waypoints} &\rightarrow \texttt{Trajectory} \rightarrow \text{modified Trajectory} \\ &\rightarrow \texttt{VehicleControl} \rightarrow \text{Unity}. \end{aligned}$$

One current interface limitation is that the decision module does not directly use the complete `TrafficLightState` enumeration. In addition, when the decision module changes trajectory velocities, it does not recompute `time_from_start`, so the timing field may become inconsistent with the modified speed profile.

3 Design Choices

3.1 Prerecorded Route Instead of Online Global Planning

Because the road-level route is known in the simulation environment, the system uses a fixed waypoint route instead of implementing an additional online global planner such as A* or RRT. The goal selector applies a monotonically increasing route index and forward search so that the vehicle can continue along the original route after a temporary deviation or local avoidance maneuver.

3.2 Current Point Cloud for Dynamic-Obstacle Detection

OctoMap [2] is retained for visualization and static-environment representation, while the hazard-monitoring module uses the current point-cloud frame directly. An accumulated occupancy map reacts slowly to moving vehicles, whereas the current point cloud responds more quickly to non-player vehicles. The disadvantage is increased sensitivity to sensor noise.

3.3 RGB-Based Traffic-Light Detection

Traffic-light detection currently uses only the RGB camera. The detector segments red, yellow, and green image regions, filters candidate contours according to size and shape, and selects the most plausible traffic-light candidate. The resulting state is accepted only after confirmation over several consecutive frames in order to reduce single-frame noise and unstable stop–go behavior.

Detection performance still depends strongly on illumination, viewpoint, image resolution, traffic-light size, and background colors.

3.4 Spline Trajectory and Constrained Velocity Profile

The local path is smoothed using a Catmull–Rom spline [3], and curvature is computed from the spline derivatives. The reference velocity is limited by the maximum vehicle speed, lateral acceleration, longitudinal acceleration, and braking constraints. This design produces a smooth and computationally inexpensive trajectory without requiring nonlinear optimization.

3.5 State-Machine-Based Safety Decision Making

The system uses a six-state decision machine for normal driving, cautious driving, local avoidance, traffic-light stopping, emergency stopping, and sensor fault handling. Emergency states can be entered immediately and remain active for a minimum duration. Normal transitions use hysteresis to reduce rapid switching.

The decision module outputs a modified trajectory rather than direct actuator commands. This preserves the modular separation between high-level behavior selection and low-level vehicle control.

3.6 Pure Pursuit and PI Control

Adaptive Pure Pursuit, based on the geometric path-tracking method described by Coulter [4], is used for lateral control because of its simple structure and suitability for waypoint tracking. The look-ahead distance increases with vehicle speed. Longitudinal control uses a PI controller [5] and previews the lowest target velocity on the upcoming trajectory segment.

When required input is missing or stale, the trajectory is empty, or the target velocity is close to zero, the controller applies full braking as a fail-safe action.

4 Results and Evaluation

4.1 Integrated Driving Demonstration

An end-to-end driving test was performed in the Unity simulation environment to evaluate the integrated behavior of the complete autonomous-driving system. The recorded test has a duration of approximately 5 min 27 s and presents the vehicle from an external third-person view. During the test, the perception, planning, decision-making, and control modules operated together while the vehicle followed the prerecorded route.

The recording demonstrates continuous route following through straight road segments, curves, intersections, and narrow passages. The vehicle encounters several NPC vehicles and continues without a visible collision during the recorded interval. However, several issues and limitations were still observed during the test; these are documented and analyzed in Section 6, *Known Code Defects and Limitations*.

Figure 3 shows a representative scene from the integrated driving test. The complete recording is available as an online demonstration video.

Figure 3: Representative scene from the integrated autonomous-driving test in Unity. The ego vehicle follows the prerecorded route in the presence of other road users.

The video provides a qualitative system-level evaluation of the externally observable vehicle behavior. It shows that the individual software components can operate together and generate continuous steering, acceleration, braking, and stopping behavior.

5 Team Responsibilities

The repository commit history does not reliably identify the author of every newly added autonomous-driving file. The following table therefore records the agreed module responsibilities during system integration and demonstration.

Member	Package/Nodes	Main Responsibility
Haitao Yu	`simulation`, `dummy_controller`, bringup	TCP/UDP bridge, coordinate conversion, simulator JSON configuration, unified launch file, and basic system integration.
Haoran Wang	`perception`	Sensor transforms, depth point cloud, OctoMap integration, RGB-based traffic-light detection, and detector debugging.
Yonglin Wang	`planning`	Route recording, route-progress estimation, local-goal selection, spline generation, and constrained velocity-profile generation.
Haodong Zhu	`decision_making`	Obstacle projection, relative closing-speed estimation, avoidance corridor, state priority, hysteresis.
Hao Ji	`control` and system evaluation	Pure Pursuit steering, PI speed control, fail-safe behavior, controller tuning.

The corresponding main ROS nodes are:

- Haitao Yu: `/Unity_ROS_message_Rx`, `/ROS_Unity_command_Tx`, `/JSON_param_reader`, and bringup.
- Haoran Wang: `/sensor_tf_broadcaster`, `/depth_to_pointcloud`, `/traffic_light_detector`, and `/octomap_server` integration.
- Yonglin Wang: `/waypoint_recorder`, `/goal_selector`, and `/trajectory_planner`.
- Haodong Zhu: `/forward_obstacle_monitor` and `/decision_state_machine`.
- Hao Ji: `/trajectory_follower` and system-evaluation topics.

6 Known Code Defects and Limitations

Defect or Limitation	Problem Description and Recommended Change
Traffic-light detection still has improvement potential	The RGB-based detector can identify most traffic lights and allows the vehicle to pass intersections according to generally logical traffic rules in most tested situations. However, small, distant, partially occluded, or turning-view traffic lights may still be missed or classified unstably. Future work should improve candidate filtering, temporal state retention, lane-to-signal association, and robustness to illumination and viewpoint changes.
No lane-to-signal association	The detector does not associate a traffic light with the ego lane, planned turn direction, or current intersection. It may therefore select a signal belonging to an adjacent lane. Stop-line, lane, and driving-direction metadata should be added.
Insufficient quantitative validation	The current documentation contains only limited online tests and targeted fault-injection tests. Repeated complete-route data have not been collected, so reliable RMS tracking error, stopping distance, system latency, and completion rate cannot yet be reported.

Defect or Limitation	Problem Description and Recommended Change
Turning-angle planning can be improved	During turning maneuvers, the controller does not always determine an appropriate steering angle sufficiently early from the upcoming road geometry. At some intersections, the selected steering angle may therefore be temporarily unreasonable or less smooth than desired. Nevertheless, the vehicle can still complete the turning task successfully. Future work should use a longer path preview, curvature-aware steering feedforward, and intersection-specific trajectory smoothing.

7 Conclusion

This project implements a modular ROS 2 autonomous-driving prototype that connects Unity sensor inputs, environmental perception, route-based planning, safety-oriented decision logic, and closed-loop vehicle control. The team has basically completed all tasks required by the project, including simulator communication, sensor-data publication, point-cloud generation, route following, trajectory planning, traffic-light and obstacle handling, decision-state management, and vehicle control.

The main design choices emphasize simplicity, real-time operation, and ease of integration. They include a prerecorded global route, current-frame point-cloud obstacle monitoring, RGB-based traffic-light detection, spline trajectory generation, a safety state machine, and Pure Pursuit/PI control. The integrated system can complete the main autonomous-driving workflow and, in most tested situations, respond to traffic lights and obstacles according to generally logical driving rules.

Some functions still have room for improvement. In particular, traffic-light detection can become unstable for distant, small, occluded, or turning-view signals. Future work should improve temporal traffic-light memory, lane-to-signal association, controller calibration, trajectory-time consistency, and repeated quantitative full-route validation.

References

[1] Open Robotics, "Ros 2 interfaces," ROS 2 Documentation, n.d., available at: https://docs.ros.org/en/rolling/Concepts/Basic/About-Interfaces.html.

[2] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3d mapping framework based on octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, April 2013.

[3] E. Catmull and R. Rom, "A class of local interpolating splines," in *Computer Aided Geometric Design*, R. E. Barnhill and R. F. Riesenfeld, Eds. New York: Academic Press, 1974, pp. 317–326.

[4] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon University, The Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-92-01, 1992.

[5] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ: Princeton University Press, 2008.